

اشتباهات کلاسیک در مهندسی نرم افزار

آرین فیروزی 810100196 - شهزاد ممیز 810100272 - علی سامانیان 810100161

بخش ۱

پرسش ۱ - چه سازوکاری در **اسکرام** **Insufficient user input** را به طور سیستماتیک رفع می‌کند؟

اسکرام بر اساس سه ستون "شفافیت"، "بازبینی" و "سازگاری" بنا شده است. در زمینه‌ی ورودی ناکافی از سمت کاربر، اسکرام از طریق مکانیزم‌های "بازبینی" (Inspection) و "سازگاری" (Adaptation) این مشکل را مدیریت می‌کند. بازبینی منظم بر روی "بک‌لاگ محصول" و نتایج بازبینی در رویدادهایی مانند "بازبینی اسپرینت" و "روزانه‌های اسکرام" (Daily Scrum) به تیم کمک می‌کند تا ورودی ناکافی را شناسایی و با استفاده از رویکرد سازگاری، تصمیمات لازم برای بهبود ورودی‌ها و تکمیل بخش‌های ناقص را اتخاذ کنند. با استفاده کردن از 4 مرحله ی planning, Daily, Review, Retro و به خصوص در گام planning وقتی که تیم اسکرام با هم و با stakeholder ها همفکری می کنند و راجب sprint event بعدی همفکری و برنامه ریزی می کنند، با پاسخ به پرسش "چرا Sprint برای stakeholder ارزشمند است". همچنین در مرحله ی Retrospective گروه اسکرام با stakeholder ها به بررسی ریزالت sprint می پردازند. انجام این گام ها باعث می شود به اولین بیانیه ی چابک مبنی بر ارائه یک محصول ارزشمند به مشتری، پاسخ دهیم و سریع و continious، دلیوری کنیم.

پرسش ۲ - چگونه روش‌های چابک با **Abandoning planning under pressure** مقابله می‌کنند؟

در اسکرام، رویدادهای مشخصی مانند "برنامه‌ریزی اسپرینت" (Sprint Planning) به عنوان بستری برای مدیریت برنامه‌ریزی در نظر گرفته شده است. با استفاده از اسکرام، تیم‌ها به طور منظم و دوره‌ای پیشرفت کار خود را بازبینی می‌کنند و در صورت تغییرات محیطی یا افزایش فشار، می‌توانند برنامه‌ها را بر اساس نیازهای جدید تطبیق دهند. این رویکرد به جای ترک برنامه‌ریزی تحت فشار، به تیم‌ها امکان می‌دهد به صورت تدریجی و با یادگیری از تجربه‌ها، برنامه‌ریزی خود را بهینه کنند. اسکرام همچنین بر مدیریت دقیق و شفاف "بک‌لاگ" تأکید دارد، که به تیم‌ها اجازه می‌دهد تحت فشار بدون از دست دادن تمرکز، تطبیق‌های لازم را انجام دهند.

برای مقابله با **Abandoning planning under pressure** مهم ترین کار این است که کار به صورت ایتريتیو و تکرار شونده انجام شود. چندین اصل مرتبط با این موضوع وجود دارد:

1- پروژه را با افراد دلسوز و با انگیزه بسازیم تا پیش برد کار برای آن ها نیز مهم باشد و به کیفیت کار اهمیت دهند.

2- اگر به عنوان یک معیار و مقیاس پیشرفت، Working Software را در اولین اولویت خود قرار دهیم، سبب میشود که فشار های ناشی را از سمت stakeholderها کمتر کنیم.

3- نرم افزار را به صورت frequent ارائه دهیم که باعث میشه وقتی یک working software را تحویل می دهیم، باعث خارج شدن فشار از سمت stake holder ها می شود و همچنین مجبور میشویم کار را به قسمت های کوچکتر بشکنیم که بسیار خوب است و برای برنامه ریزی ما بهتر است.

پرسش ۳ - کدام یک از ۱۲ اصل توسعه چابک بر پرهیز از "Wasting time in the fuzzy front end" تأکید دارد؟

بنا بر اصل 1 باید رضایت مشتری توسط تحویل سریع محصول جلب شود، پس نباید وقت زیادی در ابتدای کار از دست بدهیم. همچنین اصل 3 نیز به کوتاه کردن چرخه ی تولید تأکید دارد که به نوعی مانع وقوع fuzzy front end میشود.

پرسش ۴ - چه بخشی از اسکرام مانع از Shortchanging quality assurance to improve development speed می شود؟

برای جلوگیری از کاهش quality assurance در اسکرام میتوان از چندین روش استفاده کرد. اولاً در تعریف DoD باید توجه کنیم که مبحث کیفیت یکی از معیار ها باشد و همچنین میتوان در تعریف هدف اسپرینت و به تبع آن در Sprint Review به quality assurance پرداخت تا دچار کاهش کیفیت محصول نشویم.

پرسش ۵ - این فهرست از اشتباهات کلاسیک چگونه بند اول بیانیه چابکی Individuals and Interactions over Processes and Tools را موجه می کند؟

فهرست اشتباهات کلاسیک استیو مک کانل، اهمیت بخشیدن به افراد و تعاملات را نسبت به فرآیندها و ابزارها در بیانیه چابکی برجسته می کند. برخی از اشتباهات مطرح شده مانند "Insufficient User Input" و "Shortchanging Quality Assurance" نشان می دهند که تمرکز صرف روی فرآیندها و ابزارها می تواند منجر به نقص هایی در پروژه شود. تأکید مک کانل بر اهمیت تعاملات انسانی (از جمله کاربران و تیم توسعه) در موفقیت پروژه، نشان می دهد که فرآیندها و ابزارها نمیتوانند جایگزین تعاملات واقعی برای رسیدگی به نیازهای تغییرپذیر شوند.

بخش ۲

پرسش ۶ - در مورد Lack of feature-creep control، به نظر می رسد منظور مک کانل بیشتر معطوف به Scope Creep است تا Feature Creep. اگر این فرض را بپذیریم، توضیح دهید در فرایند اسکرام چگونه می توانیم با این مشکل برخورد کنیم. به این منظور درباره عوامل این مشکل تحقیق کنید (با ذکر منابع) و برای هر کدام راه حلی ذکر کنید. این راه حل ها می توانند از انواع زیر باشند:

- یک (یا چند) جزء از اسکرام این عامل را رفع می کنند
- لازم است تیم علاوه بر فرایند اسکرام، فعالیتهایی را برای رفع این عامل به کار گیرد
- رفع عامل مذکور در محدوده ای فراتر از تیم تولید نرم افزار صورت می گیرد

- اساساً در توسعه چابک این عامل به عنوان مشکل مطرح نمی‌شود
برای توجیه راه‌حلهایی که ارائه کرده‌اید مختصراً استدلال کنید.
اگر روش‌های چابک را مبنای کار خود داشته باشیم، راه‌حل مک‌کانل برای حل مشکل را چگونه ارزیابی می‌کنید؟
برای این که حسی از فرایندهای کنترل تغییر داشته باشید می‌توانید [مدیریت درخواست تغییر](#) از فرایند RUP را مطالعه نمایید.

1. Sprint Planning Meetings:

در هر اسپرینت، تیم با مالک محصول نیازمندی‌های مهم را انتخاب و اولویت‌بندی می‌کند. این فرایند به حفظ تمرکز روی اهداف اصلی کمک کرده و از اضافه شدن ویژگی‌های غیرضروری جلوگیری می‌کند.

2. Product Owner:

یکی از مسئولیت‌های مالک محصول، مدیریت بک‌لاگ و ارزش‌دهی به وظایف بر اساس نیازهای ذی‌نفعان است. او به‌طور منظم بک‌لاگ را بررسی کرده و اطمینان می‌دهد که توسعه‌دهندگان تنها روی ویژگی‌های ضروری و توافق‌شده تمرکز کنند.

3. Sprint Review Meetings:

این جلسات بازخورد ذی‌نفعان را دریافت و بر این اساس بک‌لاگ را به‌روزرسانی می‌کنند. این رویکرد از گسترش دامنه‌ی پروژه به‌صورت ناگهانی جلوگیری کرده و به بهینه‌سازی محصول مطابق با نیازهای جدید کمک می‌کند.

4. Change Management Techniques:

برخی ابزارهای کنترل تغییر مانند ماتریس تأثیر تغییرات و برنامه‌ریزی متوالی نیز می‌توانند در کنار اسکرام استفاده شوند تا کنترل بهتری روی تغییرات داشته باشیم.

منابع: [Avoiding classic mistakes](#) | [Scrum](#)

پرسش ۷ - در مورد Shortchanging upstream activities این طور ذکر شده که تیم‌ها تحت فشار زمانی فعالیت‌هایی مثلاً تحلیل نیازمندی‌ها، طراحی و معماری را حذف می‌کنند. سؤالی که در این باره پیش می‌آید این است که روش‌های چابک با توجه به تأکید بر تحویل مکرر و زود هنگام چه میزان روی طراحی توان صرف می‌کنند. به این منظور مقاله [Is Design Dead](#) را بخوانید^۱ و بعد درباره این که آیا ممکن است اشتباه مورد نظر مک‌کانل در روش‌های چابک نیز اتفاق بیفتد بحث کنید.

در مقاله "Is Design Dead" نویسنده به بررسی اختلاف بین طراحی پیش‌بینی‌شده و طراحی تکاملی در چارچوب برنامه‌نویسی چابک می‌پردازد. مک‌کانل معتقد است که در شرایط فشار زمانی، تیم‌ها ممکن است فعالیت‌های بالا دستی مانند تحلیل نیازمندی‌ها، طراحی و معماری را کنار بگذارند. این خطر در تیم‌هایی که از روش‌های سنتی استفاده می‌کنند وجود دارد، زیرا آن‌ها طراحی را در یک مرحله خاص متمرکز می‌کنند.

در مقابل، روش‌های چابک بر طراحی تکاملی و تدریجی تأکید دارند. این روش‌ها با استفاده از تحویل مکرر و بازخوردهای پیوسته، طراحی را نه به‌عنوان یک مرحله اولیه و ایستا، بلکه به‌عنوان یک فعالیت پویا و همیشگی در نظر می‌گیرند. بنابراین، به جای کنار گذاشتن طراحی، چابک طراحی را به طور مداوم در طول چرخه توسعه اصلاح و بهبود می‌دهد.

فاولر معتقد است که در روش‌های چابک، با وجود تأکید بر تحویل سریع، خطای مک‌کانل کمتر اتفاق می‌افتد. زیرا طراحی همیشه بهبود می‌یابد و با نیازمندی‌های جدید سازگار می‌شود. این طراحی مستمر و تکاملی در اسکرام و XP از طریق بازبینی‌های مکرر و تطابق با تغییرات محیطی به خوبی پشتیبانی می‌شود. بنابراین، فشار زمانی در روش‌های چابک منجر به حذف طراحی نمی‌شود، بلکه تیم‌ها را به سمت بازبینی و بهبود مستمر طراحی سوق می‌دهد.

در نهایت، هرچند ممکن است در برخی تیم‌ها خطای "shortchanging upstream activities" رخ دهد، اما اصول چابک به طور طبیعی با این مشکل مقابله می‌کنند و طراحی را به جای حذف، تقویت می‌کنند.

پرسش ۸ - چه نمونه‌هایی از Silver-bullet syndrome سراغ دارید که در سال‌های اخیر مطرح و تبلیغ شده باشند؟

این اصطلاح به راهکارهای ساده‌ای که برای مشکلات پیچیده ارائه میشوند اطلاق میشود که اغلب این قبیل محصولات و ایده‌ها با شکست مواجه شده‌اند، اما در برخی موارد (مثل واکسن پنی سیلین) موفق عمل کرده‌اند.

برای مثال فناوری ویرایش ژن CRISPR در حوزه بیوتکنولوژی به عنوان ابزاری قدرتمند برای درمان بیماری‌های ژنتیکی، بهبود محصولات کشاورزی و حتی افزایش طول عمر انسان مطرح شده است. اما کاربردهای عملی آن هنوز در مراحل اولیه است و مسائل اخلاقی و ایمنی زیادی در مورد استفاده از این فناوری وجود دارد که حل نشده‌اند.

همچنین در حوزه نرم افزار، NCDP ها به عنوان راه حلی برای توسعه نرم افزار بدون دانش کافی برای کد نویسی مطرح شدند ولی به علت محدودیت های فنی و عدم وجود scalability برای همه ی نیاز های موجود پاسخگو نبودند.

راهکار های agile و فریمورک هایی مثل scrum به عنوان راه حل کامل برای بهبود بهره وری و تولید محصول ارائه شد اما برای همه ی بیزینس ها کاربرد ندارد و استفاده ی نادرست و افراطی از آن به نتایج معکوس منجر میشود. برای مثال این متد برای حفظ تمیزی کد راه حلی ارائه نمیدهد.